



kubernetes

VOLUMES AND NETWORKING

Understanding Kubernetes & Volumes: The Big Picture

- **Why Volumes Matter:** Kubernetes provides sophisticated options for data management to support app persistence, ensuring state and data continuity.
- **Kubernetes vs. Docker Volumes:** Unlike Docker, Kubernetes offers various volume types, each suited to different persistence needs.
- **Data Durability & Portability:** Kubernetes volumes make it easy to manage data in a distributed, ephemeral environment.



Kubernetes can mount Volumes into Containers

A broad variety of Volume types / drivers are supported

"Local" Volumes (i.e. on Nodes)

Cloud-provider specific Volumes

Volume lifetime depends on the Pod lifetime

Volumes survive Container restarts (and removal)

Volumes are removed when Pods are destroyed

Exploring Kubernetes

Volume Types

Kubernetes supports several types of Volumes but here are once which are important to understand:

- ***emptyDir***: Ideal for temporary data storage within Pods; gets erased if the Pod restarts.
- ***hostPath***: Links directly to a specific directory on the node, suitable for cases requiring host-level access.
- ***CSI (Container Storage Interface)***: Allows using various storage solutions (e.g., AWS EBS, Azure Disk) in a standardized way.



Introduction to Persistent Volumes (PVs)

Persistent Data Made Easy: With PVs, data can remain accessible and reliable even as Pods are re-deployed.

- **What is Persistent Volumes:** Enables storage that outlives Pods, essential for data consistency across restarts.
- **Persistent Volumes vs. Volumes:** Regular volumes are Pod-bound, while PVs are cluster-wide resources.
- **Role of Storage Classes:** Simplifies defining PVs by specifying storage requirements for different environments (e.g., SSD, HDD).

```
! host-pv.yaml > {} spec > {} hostPath
```

```
1  apiVersion: v1
2  kind: PersistentVolume
3  metadata:
4    | name: host-pv
5  spec:
6    | capacity:
7    |   storage: 1Gi
8    | volumeMode: Filesystem
9    | accessModes:
10   |   - ReadWriteOnce
11   | hostPath:
12   |   path: /data
13   |   type: DirectoryOrCreate
```

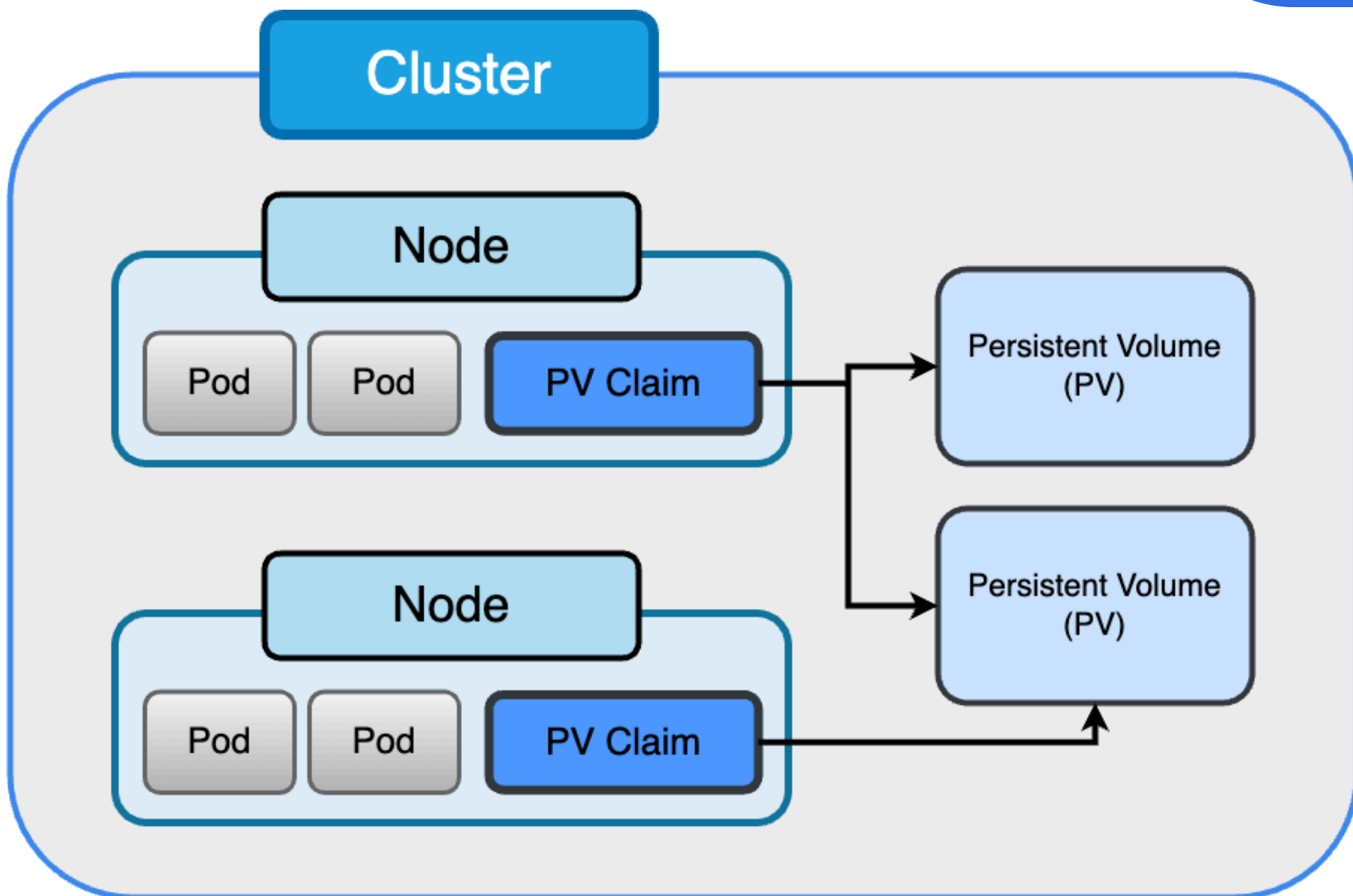
Claiming Persistent Storage with PVCs

- **What's a PVC:** Persistent Volume Claims allow Pods to request storage resources dynamically.
- **Linking PVCs to PVs:** PVCs create a bridge, letting Kubernetes dynamically provision storage for Pods.
- **Advantages:** Flexibility, dynamic scaling, and data retention across Pod restarts.
- **Best Practices:** Define specific storage needs in PVCs to optimize resource allocation.



! host-pvc.yaml > {} spec > [] accessModes

```
1  apiVersion: v1
2  kind: PersistentVolumeClaim
3  metadata:
4    | name: host-pvc
5  spec:
6    | volumeName: host-pv
7    | accessModes:
8    | | - ReadWriteOnce
9    | resources:
10   | | requests:
11   | | | storage: 1Gi
```

Managing Configurations with Environment Variables & ConfigMaps

- **Purpose of ConfigMaps:** Separate configuration from code, keeping app settings manageable and centralized.
 - **Environment Variables in Kubernetes:** Pass critical information to containers without hardcoding it into the image.
 - **Using ConfigMaps with Pods:** Attach ConfigMaps to Pods to inject configuration data dynamically.
- Best Practices:** Use ConfigMaps for non-sensitive data, and secrets for sensitive information.

Kubernetes Networking

Pod-internal Communication

- **Communication via localhost:** Containers within the same Pod interact through localhost.
- **Shared Network Namespace:** All containers share the same IP and ports.
- **Direct Communication:** No need for external networking tools.
- **Optimized Data Exchange:** Suitable for lightweight, tightly-coupled components.
- **Use Case:** Works well for apps with sidecar containers (e.g., proxies or loggers).

Tied Lifecycle: Containers are co-located and share resources, starting and stopping together.

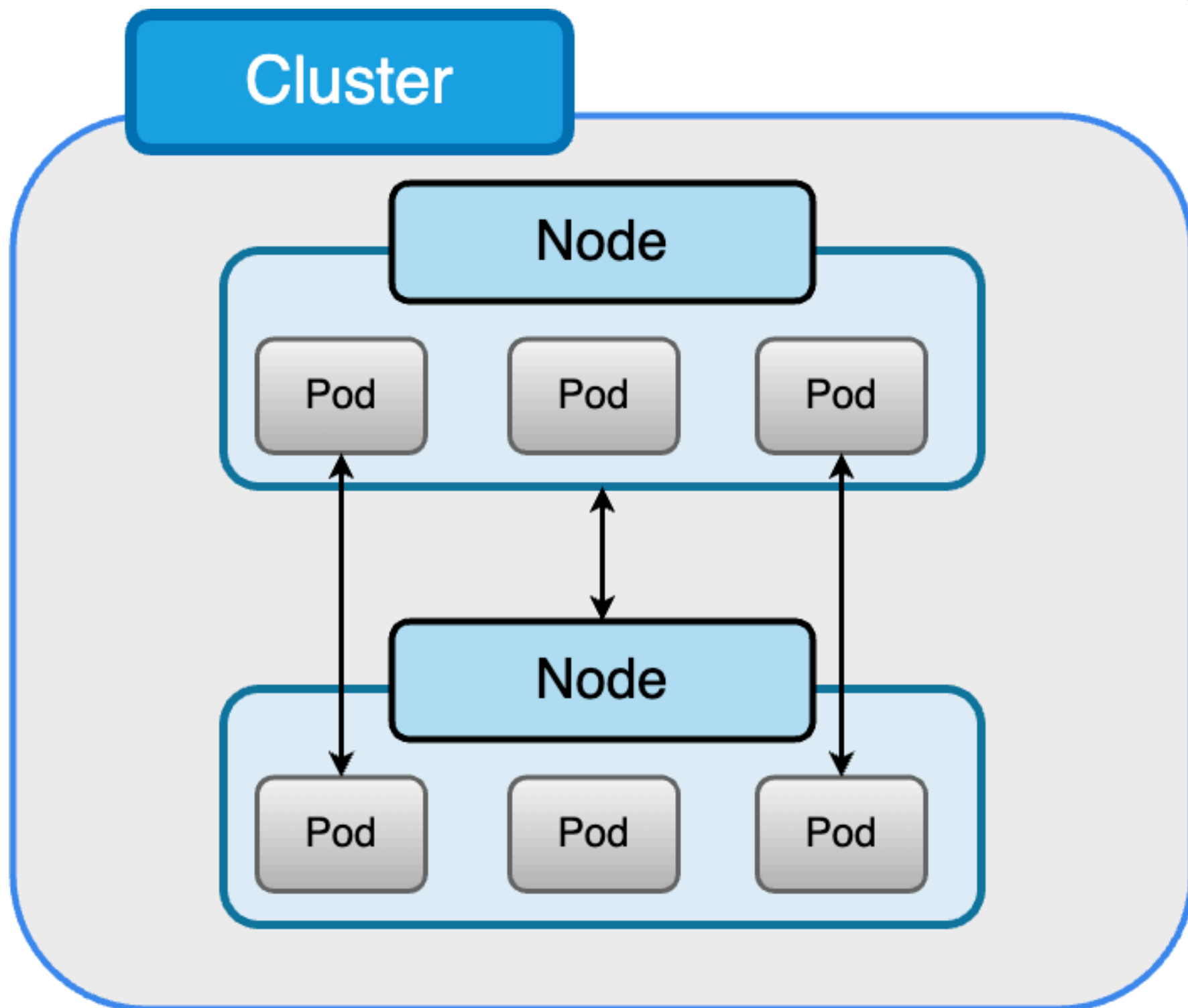
ClusterIP

Communication

- **Internal Access Only:** ClusterIP services are accessible only within the Kubernetes cluster.
- **Stable IP Address:** Assigns a virtual IP for consistent communication between services.
- **No External Exposure:** Services remain hidden from outside users.
- **DNS Support:** Pods can connect using service names instead of IPs.

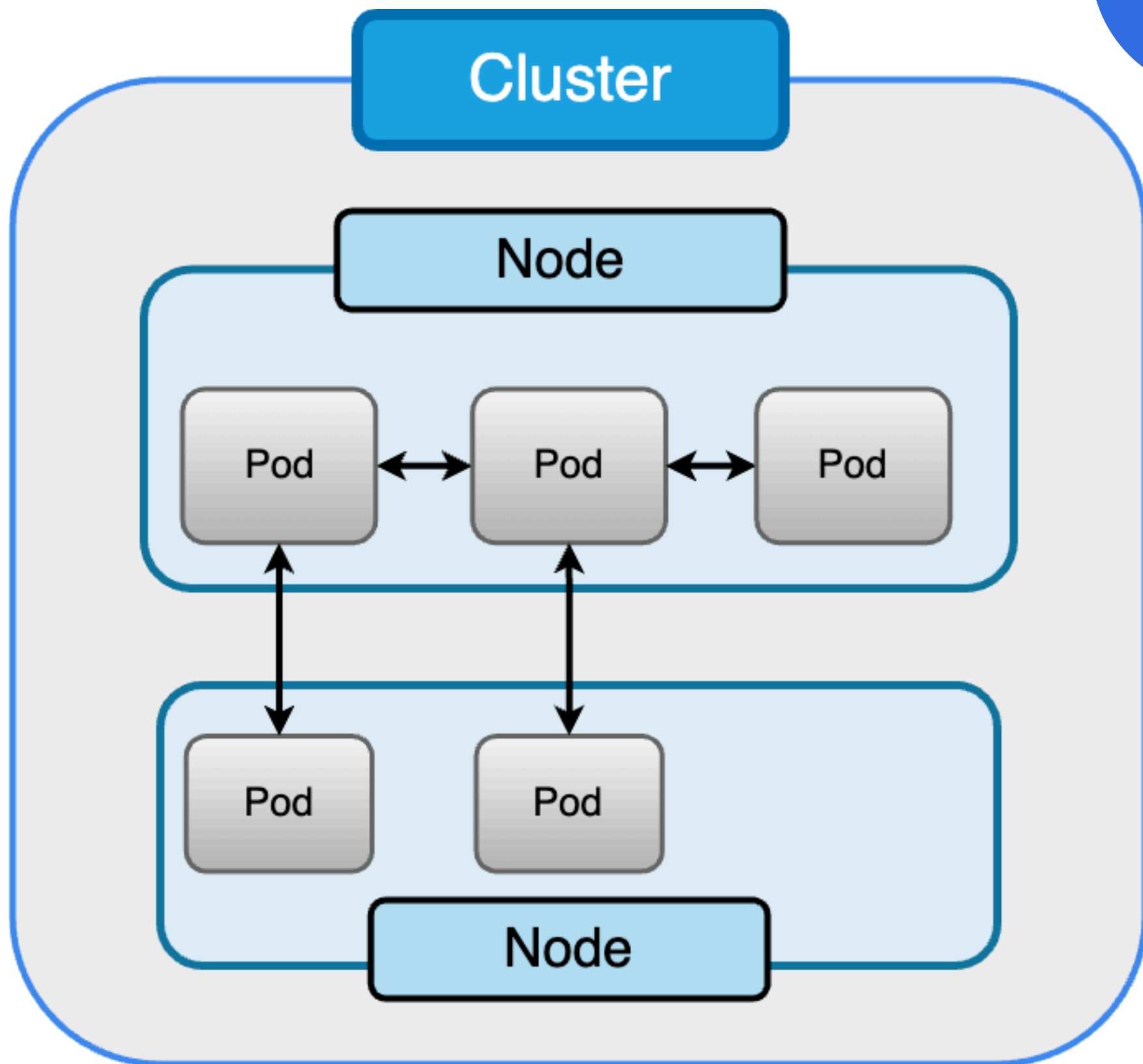
Use Case: Common for backend components like databases.

Cluster



Pod DNS Communication

- **Overview:**
- Pod-to-Pod communication in Kubernetes can be achieved through **DNS names** or **IP addresses**, allowing seamless interaction between microservices.
- **DNS-Based Communication:**
- Each service gets a **DNS name** (e.g., service-name.namespace.svc.cluster.local), enabling Pods to access services without hard-coding IPs.
- DNS records automatically update with IP changes, ensuring reliable connectivity.
- **IP-Based Communication:**
- Pods can directly connect using IP addresses, suitable for scenarios requiring low-level networking.
- However, this method requires tracking changes, as IPs can vary when Pods restart or are rescheduled.
- **Environment Variables:**
- Kubernetes injects connection details into Pods, providing both DNS and IP information for easy access.



Pod to Pod communication

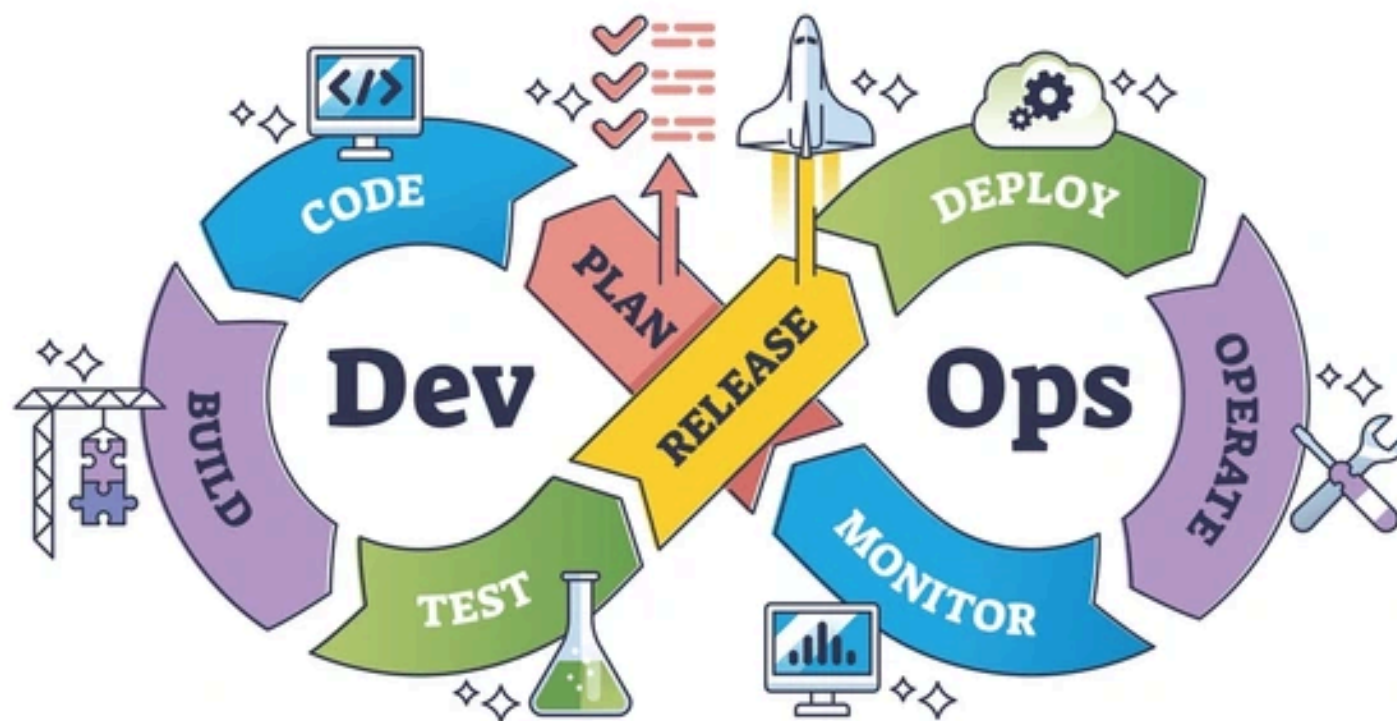
Using a Reverse Proxy

A reverse proxy is an intermediary that routes client requests to backend services, enhancing security and performance.

Key Functions:

- **Load Balancing:** Distributes traffic across multiple backend services to prevent overload.
- **SSL Termination:** Offloads SSL decryption, simplifying security management.
- **Caching:** Stores frequently accessed responses to improve load times.
- **Security:** Hides internal services, providing a single point of entry to mitigate threats.
- **Flow of Requests:**

Client requests are sent to the Reverse Proxy, which then forwards them to the appropriate Backend Services, returning responses back to the Client.



Thank You for Joining My Kubernetes
Journey! ✨ ✨